

DoRA: Báo cáo cài đặt và tái hiện thực nghiệm

Phùng Hữu Khoa¹ Lê Tiến Nghĩa¹ Lê Tuấn Duy¹

Abstract

Báo cáo này trình bày quá trình cài đặt lại và tái hiện một phần phương pháp Weight-Decomposed Low-Rank Adaptation (DoRA) của Liu và cộng sự. Thay vì tái hiện toàn bộ hệ thí nghiệm gốc trên LLaMA, LLaVA và VL-BART, chúng tôi tập trung vào thuật toán lõi: đóng băng mô hình tiền huấn luyện, thay thế các lớp tuyến tính mục tiêu bằng các mô-đun LoRA hoặc DoRA tự cài đặt bằng PyTorch, sau đó đánh giá trên mô hình `vinai/phobert-base-v2`. Thực nghiệm chính được chạy trên tập UIT-VSFC với ba seed 42, 43 và 44, so sánh tinh chỉnh toàn phần, LoRA và DoRA ở hạng 8 và 16. Kết quả cho thấy các phương pháp PEFT đạt độ chính xác xấp xỉ tinh chỉnh toàn phần: full fine-tuning đạt accuracy trung bình 0.9478, LoRA r16 đạt 0.9480 và DoRA r16 đạt 0.9478. Trong khi đó, LoRA/DoRA chỉ huấn luyện dưới 1% số tham số và tạo checkpoint khoảng 3.4–4.6 MB, nhỏ hơn rất nhiều so với checkpoint full fine-tuning khoảng 5.1 GB. Báo cáo cũng chỉ ra các khác biệt quan trọng so với bài báo gốc, các quyết định cài đặt, và những hạn chế khiến kết quả không thể được hiểu là tái hiện đầy đủ toàn bộ công bố DoRA.

1. Giới thiệu

Tinh chỉnh mô hình ngôn ngữ tiền huấn luyện là cách phổ biến để thích ứng mô hình sang một tác vụ cụ thể. Tuy nhiên, tinh chỉnh toàn phần yêu cầu cập nhật toàn bộ tham số, lưu trữ một checkpoint lớn cho từng tác vụ và tiêu tốn nhiều bộ nhớ GPU trong huấn luyện. Các phương pháp tinh chỉnh hiệu quả tham số (parameter-efficient fine-tuning, PEFT) được phát triển để giảm chi phí này bằng cách chỉ huấn luyện một phần nhỏ tham số bổ sung.

Trong nhóm PEFT, Low-Rank Adaptation (LoRA) (Hu et al., 2022) là một phương pháp quan trọng vì có thiết

¹Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội. Correspondence to: Phùng Hữu Khoa <khoa555.rubik@gmail.com>.

kế đơn giản, không thay đổi kiến trúc suy luận sau khi gộp trọng số, và dựa trên giả thuyết rằng cập nhật khi fine-tuning có hạng nội tại thấp (Aghajanyan et al., 2021). DoRA (Liu et al., 2024) tiếp tục mở rộng LoRA bằng cách tách trọng số thành thành phần độ lớn và hướng, nhằm mô phỏng gần hơn hành vi học của tinh chỉnh toàn phần trong khi vẫn giữ số tham số huấn luyện thấp.

Mục tiêu là cài đặt lại các thành phần cốt lõi của LoRA và DoRA trên `torch.nn.Linear`, tích hợp vào PhoBERT, chạy lại các thí nghiệm so sánh với full fine-tuning, và phân tích mức độ tái hiện được trong một bối cảnh nhỏ hơn so với bài báo gốc. Phạm vi tái hiện là một phần: chúng tôi không chạy lại các benchmark LLaMA, LLaVA, VL-BART của Liu và cộng sự, mà kiểm chứng thuật toán trên `vinai/phobert-base-v2` với tập UIT-VSFC.

2. Tóm lược bài báo gốc

2.1. Low-Rank Adaptation

LoRA giữ nguyên trọng số tiền huấn luyện $W_0 \in \mathbb{R}^{d \times k}$ và chỉ học một cập nhật hạng thấp:

$$W' = W_0 + \Delta W = W_0 + BA, \quad (1)$$

trong đó $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ và $r \ll \min(d, k)$. Trong triển khai thường dùng, A được khởi tạo ngẫu nhiên, B được khởi tạo bằng 0, và cập nhật được nhân với hệ số α/r . Sau huấn luyện, BA có thể được gộp vào W_0 , nên mô hình suy luận không cần thêm lớp phụ.

2.2. Weight-Decomposed Low-Rank Adaptation

DoRA bắt đầu từ nhận xét rằng LoRA và full fine-tuning tạo ra các mẫu thay đổi khác nhau về độ lớn và hướng của trọng số. Với trọng số W , DoRA biểu diễn:

$$W = m \frac{V}{\|V\|_c}, \quad (2)$$

trong đó m là thành phần độ lớn và $V/\|V\|_c$ là thành phần hướng đã chuẩn hoá theo từng vector cột hoặc hàng, tùy quy ước triển khai. Trong báo cáo này, do trọng số của `torch.nn.Linear` có dạng `[out_features, in_features]`, chúng tôi chuẩn hoá theo từng hàng đầu ra:

$$W = m \frac{W_0 + BA}{\|W_0 + BA\|}. \quad (3)$$

Như vậy, DoRA học riêng vector độ lớn m và dùng cập nhật LoRA để điều chỉnh hướng. Bài báo gốc cho thấy DoRA thường vượt LoRA trên nhiều mô hình và tác vụ, nhưng kết quả đó được báo cáo trên các thiết lập lớn hơn nhiều so với phạm vi tái hiện của báo cáo này.

2.3. Điểm chưa tái hiện

Bài báo gốc đánh giá DoRA trên các mô hình LLaMA, LLaVA và VL-BART, bao gồm các tác vụ suy luận thông thường, instruction tuning, và hiểu ảnh/video-văn bản. Do hạn chế phần cứng và phạm vi dự án, báo cáo này chỉ tái hiện thuật toán lỗi và kiểm chứng xu hướng trên PhoBERT. Vì vậy, kết quả của chúng tôi chỉ phản ánh tính đúng đắn và hiệu quả tương đối của cài đặt trong bối cảnh nhỏ, không phải bản tái hiện đầy đủ mọi bảng kết quả của công bố gốc.

3. Chi tiết cài đặt

3.1. Môi trường và phần cứng

Mã nguồn được chạy tại commit c4d52f8. Môi trường phần mềm gồm Python 3.13.3, PyTorch 2.8.0+cu129, Transformers 4.57.6, Datasets 3.6.0, scikit-learn 1.7.0 và pandas 2.3.0. Phần cứng dùng cho benchmark là NVIDIA GeForce RTX 3060 với 12 GB VRAM, driver 581.80.

3.2. Mô hình và dữ liệu

Backbone sử dụng trong toàn bộ thí nghiệm là `vinai/phobert-base-v2`. Thí nghiệm chính dùng tập UIT-VSFC tải qua Hugging Face với ba nhãn cảm xúc. Dữ liệu được token hoá bằng tokenizer của PhoBERT, cắt ngắn ở độ dài tối đa 256 token. Nếu tập dữ liệu không có validation split, mã nguồn có cơ chế tách 10% train thành validation, nhưng trong benchmark chính chúng tôi dùng split có sẵn của UIT-VSFC.

Repo cũng có pipeline tạo dữ liệu ESG đa nhãn từ nhãn yếu, gồm 2,439 mẫu, 10 nhãn và các split train/validation/test tương ứng 2,043/194/202 mẫu. Tuy nhiên, chưa có benchmark ESG đầy đủ trong bảng kết quả chính; do đó phần này chỉ được xem là hướng mở rộng của cài đặt, không phải kết quả tái hiện chính.

3.3. Cài đặt LoRA và DoRA

Phần cài đặt được viết theo hướng thay thế trực tiếp các lớp `torch.nn.Linear`, thay vì dùng thư viện PEFT có sẵn. Với LoRA, lớp `LoRALinear` sao chép trọng số và bias của lớp tuyến tính gốc sang các tham số đóng băng, sau đó thêm hai ma trận trainable `lora_A` và `lora_B`. Ma trận `lora_A` được khởi tạo bằng Kaiming uniform, còn `lora_B` được khởi tạo bằng 0 để tại bước đầu tiên nhánh adapter chưa làm thay đổi output của mô hình tiền huấn luyện. Trong forward pass, module tính nhánh gốc

Table 1. Khác biệt cài đặt giữa FT, LoRA và DoRA.

Chế độ	Tham số huấn luyện	Module thay	Checkpoint
FT	Toàn bộ mô hình	Không	Full model
LoRA	Classifier + A, B	<code>query, value</code>	Trainable
DoRA	Classifier + m, A, B	<code>query, value</code>	Trainable

$F(x; W_0)$ và nhánh cập nhật hạng thấp $F(F(x; A); B)$, rồi cộng hai nhánh với hệ số α/r . Khi cần suy luận, trọng số có thể được gộp thành $W_0 + (BA)\alpha/r$ và chuyển lại thành một lớp `nn.Linear` thông thường.

Với DoRA, lớp `DoRALinear` cũng bọc một lớp tuyến tính gốc, nhưng không cộng trực tiếp cập nhật hạng thấp vào output như LoRA. Trọng số gốc W_0 được lưu trong buffer `weight_base`; vector magnitude được khởi tạo bằng norm của từng hàng trọng số gốc và được huấn luyện cùng `lora_A, lora_B`. Do trọng số của `torch.nn.Linear` có dạng `[out_features, in_features]`, cài đặt hiện tại chuẩn hoá theo từng hàng:

$$\tilde{W} = m \frac{W_0 + (BA)\alpha/r}{\|W_0 + (BA)\alpha/r\|_2}. \quad (4)$$

Forward pass của DoRA dùng trực tiếp $\tilde{W}$ trong `F.linear`. Điểm khác biệt quan trọng là LoRA học một nhánh cộng thêm vào output của lớp gốc, còn DoRA tái tạo trọng số hiệu dụng từ thành phần độ lớn và hướng ở mỗi forward pass.

Việc tích hợp vào PhoBERT được thực hiện bởi hàm `apply_peft`. Với chế độ `full fine-tuning`, toàn bộ tham số mô hình được đặt `requires_grad=True`. Với LoRA/DoRA, mã nguồn trước hết gọi `freeze_all_but_classifier()` để đóng băng backbone và giữ classifier head trainable; sau đó duyệt `model.named_modules()`, tìm các lớp `nn.Linear` có tên khớp với `target_modules`, và thay chúng bằng `LoRALinear` hoặc `DoRALinear`. Trong benchmark chính, `target_modules` là `query, value`, tương ứng với các projection query và value trong self-attention của 12 tầng encoder PhoBERT; tổng cộng 24 lớp tuyến tính được thay thế. Quá trình huấn luyện sau đó vẫn dùng Hugging Face Trainer: dữ liệu được token hoá, mô hình được train/evaluate theo từng epoch, VRAM đỉnh và thời gian huấn luyện được đo, rồi kết quả được append vào `results/benchmark_results.csv`.

4. Thiết lập thí nghiệm

Chúng tôi chạy năm biến thể: full fine-tuning (FT), LoRA r8, LoRA r16, DoRA r8 và DoRA r16. Mỗi biến thể được chạy với ba seed 42, 43 và 44. Các độ đo chính gồm accuracy, macro-F1 và weighted-F1. Ngoài chất lượng phân loại, báo cáo ghi nhận số tham số huấn luyện, tỉ lệ tham số huấn luyện,

Table 2. Siêu tham số chính trong các thí nghiệm.

Thiết lập	Giá trị
Backbone	vinai/phobert-base-v2
Target modules	query, value
Epochs	3
Batch size	16
Eval batch size	32
Max length	256
Seeds	42, 43, 44
LoRA/DoRA ranks	8, 16
Alpha	$2r$
Dropout	0.05
Learning rate FT	2×10^{-5}
Learning rate PEFT	2×10^{-4}

thời gian huấn luyện, VRAM đỉnh và kích thước checkpoint.

Trong file `results/benchmark_results.csv`, ba dòng DoRA r8 đầu tiên của seed 42 có thời gian hoặc VRAM bất thường và xuất hiện trước chuỗi benchmark đầy đủ. Chúng tôi xem đây là các smoke/debug runs và loại khỏi bảng chính. Bảng kết quả chỉ dùng các run từ `ft_seed42_1780320959` trở đi, tức mỗi biến thể có đúng ba seed.

5. Kết quả tái thực nghiệm và so sánh

Bảng 3 cho thấy LoRA và DoRA đạt accuracy rất gần full fine-tuning trong thiết lập PhoBERT/UIT-VSFC. LoRA r16 có accuracy trung bình cao nhất, nhưng chênh lệch với FT và DoRA r16 rất nhỏ. Về macro-F1, full fine-tuning vẫn tốt hơn các biến thể PEFT; điều này cho thấy các cập nhật tham số thấp có thể giữ được độ chính xác tổng thể nhưng chưa chắc tối ưu bằng FT trên từng lớp nhãn.

Lợi ích rõ nhất của LoRA/DoRA nằm ở chi phí huấn luyện và lưu trữ. FT cập nhật toàn bộ 135 triệu tham số và tạo checkpoint khoảng 5.1 GB, trong khi các biến thể PEFT chỉ huấn luyện dưới 1% tham số và checkpoint chỉ khoảng 3.4–4.6 MB. VRAM đỉnh của PEFT cũng thấp hơn đáng kể, khoảng 1.84–1.86 GB so với 3.18 GB của FT.

6. Phân tích và thảo luận

Mức độ tái hiện. Báo cáo tái hiện thành công ý tưởng cốt lõi của DoRA: tách độ lớn và hướng của trọng số, dùng cập nhật hạng thấp cho thành phần hướng, và chỉ huấn luyện một lượng nhỏ tham số. Các kiểm thử trong repo cũng xác nhận wrapper LoRA/DoRA có thể thay thế lớp tuyến tính và tạo checkpoint adapter nhỏ.

So sánh xu hướng với bài báo gốc. Trong bài báo gốc, DoRA thường vượt LoRA trên nhiều tác vụ lớn. Trong

thí nghiệm của chúng tôi, DoRA không vượt LoRA một cách nhất quán trên macro-F1 hoặc accuracy. Đây không phải mâu thuẫn trực tiếp với bài báo gốc vì thiết lập đã thay đổi đáng kể: mô hình là PhoBERT-base thay vì LLM/LVLM lớn, dữ liệu là UIT-VSFC, số epoch nhỏ, và chỉ thay `query, value`. Kết quả nên được hiểu là DoRA đạt hiệu quả tài nguyên tương tự LoRA và gần FT trong bối cảnh này, chứ chưa chứng minh ưu thế tổng quát của DoRA.

Khó khăn và hướng giải quyết. Một số chi tiết của paper gốc không được tái hiện đầy đủ, chẳng hạn lịch siêu tham số cho từng benchmark lớn, cấu hình mô hình LLaMA/LLaVA/VL-BART và các phép đánh giá chuyên biệt. Báo cáo vì vậy chọn hướng tái hiện thuật toán lõi với Hugging Face Trainer. Optimizer và scheduler dùng mặc định của framework, trong khi learning rate, batch size, rank, alpha và dropout được cố định trong mã nguồn để đảm bảo dễ lặp lại.

7. Kết luận

Báo cáo đã chuyển trọng tâm từ việc trình bày một phương pháp mới sang mô tả quá trình cài đặt và tái hiện một phần DoRA. Chúng tôi đã cài đặt LoRA và DoRA từ đầu cho `torch.nn.Linear`, tích hợp vào PhoBERT, và so sánh với full fine-tuning trên UIT-VSFC. Kết quả cho thấy các biến thể PEFT đạt accuracy gần tương đương FT trong khi chỉ dùng dưới 1% tham số huấn luyện và giảm mạnh kích thước checkpoint. Tuy nhiên, DoRA chưa thể hiện ưu thế rõ ràng so với LoRA trong thiết lập nhỏ này. Hướng tiếp theo là chạy benchmark đầy đủ trên dữ liệu ESG đa nhãn, bổ sung ablation theo target modules, và nếu có tài nguyên, tái hiện thêm một phần benchmark gốc của DoRA trên mô hình lớn hơn.

Tài liệu

Aghajanyan, A., Gupta, S., and Zettlemoyer, L. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 2021.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., and Chen, M.-H. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.

Table 3. Kết quả benchmark trên UIT-VSFC. Giá trị được báo cáo theo trung bình $\pm$ độ lệch chuẩn trên ba seed.

Phương pháp	Accuracy	Macro-F1	Weighted-F1	Tham số train	Trainable %	VRAM MB	Checkpoint MB
FT	0.9478 ± 0.0004	0.8532 ± 0.0033	0.9455 ± 0.0001	135,000,579	100.0000	3180.6 ± 4.9	5158.3
LoRA r8	0.9459 ± 0.0028	0.8356 ± 0.0093	0.9422 ± 0.0031	887,811	0.6562	1841.5 ± 9.0	3.4
LoRA r16	0.9480 ± 0.0016	0.8403 ± 0.0112	0.9445 ± 0.0021	1,182,723	0.8723	1846.8 ± 9.0	4.5
DoRA r8	0.9467 ± 0.0010	0.8330 ± 0.0058	0.9429 ± 0.0005	906,243	0.7480	1854.1 ± 7.3	3.5
DoRA r16	0.9478 ± 0.0004	0.8390 ± 0.0093	0.9445 ± 0.0008	1,201,155	0.9890	1857.4 ± 7.3	4.6

A. Hướng dẫn chạy lại

Cài đặt phụ thuộc:

```
pip install -r requirements.txt
```

Chạy full fine-tuning:

```
python train.py --method ft --dataset uit-vsfc --seed 42
```

Chạy LoRA:

```
python train.py --method lora --rank 8 --alpha 16 --dropout 0.05 --seed 42  
python train.py --method lora --rank 16 --alpha 32 --dropout 0.05 --seed 42
```

Chạy DoRA:

```
python train.py --method dora --rank 8 --alpha 16 --dropout 0.05 --seed 42  
python train.py --method dora --rank 16 --alpha 32 --dropout 0.05 --seed 42
```

B. Dữ liệu ESG đa nhãn

Pipeline mở rộng cho dữ liệu ESG được tạo bằng lệnh:

```
python scripts/build_multilabel_dataset.py \  
    --input-dir extracted_data/extracted_labels_both \  
    --output-dir data/multilabel
```

Tập dữ liệu thu được có 2,439 mẫu, gồm 2,043 mẫu train, 194 mẫu validation và 202 mẫu test. Có 10 nhãn: E1, E2, E3, E_COMMON, G10, G11, G9, S4, S5 và S6. Vì chưa có kết quả huấn luyện đầy đủ trong bảng benchmark chính, phần này chưa được dùng để đưa ra kết luận thực nghiệm.

C. Các run bị loại khỏi bảng chính

Ba dòng DoRA r8 seed 42 đầu tiên trong `results/benchmark_results.csv` không được dùng trong Bảng 3: `dora_r8_seed42_1780318896`, `dora_r8_seed42_1780320141`, và `dora_r8_seed42_1780320630`. Lý do là các dòng này xuất hiện trước chuỗi benchmark đầy đủ và có dấu hiệu không đồng nhất với các run chính, chẳng hạn thời gian huấn luyện hoặc VRAM bất thường.